



JMPM

AdvPP - Compilador AdvPL/TLPP em Go

DATA
05/09/2026

ARQUIVO
README.md



Um compilador e interpretador totalmente funcional para as linguagens de programação AdvPL e TLPP, construído em Go.

Instalação

```
curl -fsSL https://raw.githubusercontent.com/peder1981/AdvPP/master/install.sh | sh
```

Detecta SO/arquitetura automaticamente (Linux amd64/arm64, macOS Apple Silicon) e instala o binário `advplc` mais recente em `~/local/bin` (ou `/usr/local/bin` como root). Sem Go, sem dependências — o binário é estático.

Alternativas:

- **Debian/Ubuntu:** baixe o `.deb` em Releases e `sudo apt install ./advpp_*.deb`
- **Windows:** baixe o `.zip` em Releases
- **Extensão VS Code** (syntax highlighting, build/run/debug, debugger real, `advplc serve` attach): baixe o `.vsix` em Releases e instale com `code --install-extension advpl-tlpp-*.vsix` — já vem com o compilador embutido (linux-x64, linux-arm64, win32-x64, darwin-arm64), nada mais pra instalar. Fonte em [tools/vscode-advpl](#)

Compilando do fonte

Requer Go 1.24+:

```
git clone https://github.com/peder1981/AdvPP && cd AdvPP
make build # gera advplc, adveditor, advpp-ide na raiz do repo
```

CI roda a suite completa em Linux, Windows e macOS a cada push

(`.github/workflows/test.yml`). Ao escrever código ou testes platform-specific (FFI, `unsafe`, arquivos temporários, etc.), leia

`docs/testing-cross-platform.md` antes —

documenta o padrão e os bugs reais já encontrados nessa categoria.

Recursos

- **Lexer:** Tokenizador completo para sintaxe AdvPL/TLPP incluindo palavras-chave, operadores, blocos de código e diretivas de pré-processador
- **Pré-processador:** Trata `#include`, `#define` (inclusive multi-linha), `#ifdef` / `#ifndef` / `#else` / `#endif`, e `#xCommand` / `#command` / `#xTranslate` / `#translate` com pattern-matching real (marcadores `<nome>`, cláusulas opcionais `[...]`, flags `[<nome:LITERAL>]`, resultado com `<{nome}>` / `<.nome.>`)
- **Parser:** Parser recursivo descendente completo gerando uma AST
- **Compilador:** Gera bytecode otimizado com 87 opcodes
- **Serialização de Bytecode:** Salva bytecode compilado em disco para execução posterior
- **Executáveis Standalone:** Constrói executáveis autossuficientes com bytecode embutido usando go:embed
- **Máquina Virtual:** VM completa com todos os opcodes implementados
- **Runtime:** Funções nativas (ConOut, MsgInfo, AllTrim, Str, Val, aAdd, aScan, Len, etc.)
- **I/O de disco, arquivo e sistema:** `MemoRead` / `MemoWrite` / `FErase`, API de handle para streaming (`FOpen` / `FCreate` / `FReadStr` / `FWrite` / `FSeek` / `FClose` / `FError`), console interativo `ConIn` e chamada de sistema `WaitRun` — ver seção Funções de I/O, arquivo e sistema
- **BLAS ternária + IA em AdvPL puro:** kernel *multiply-free* `MatVecTern` (produto matriz-vetor ternário estilo BitNet) e três modelos escritos inteiramente em AdvPL — Markov (`pt_llm`), respondedor por recuperação (`pt_chat`) e híbrido Markov+rede neural ternária (`pt_nn`); ver [Exemplos de IA em AdvPL puro](#)
- **IDE Gráfica:** Ambiente de Desenvolvimento Gráfico usando Fyne com editor de código, navegador de arquivos e compilador integrado
- **Framework UI:** Aplicações gráficas usando Fyne (diálogos, formulários, grids, botões, menus)
- **Banco de Dados:** Operações de banco de dados baseadas em Workarea (DbSelectArea, DbSeek, DbSkip, RecLock, etc.)
- **Classes:** Sistema de classes completo com Data/Method/Constructor, herança via `from`
- **Blocos de Código:** Blocos de código executáveis `{|| ... }`
- **MVC:** Suporte FWFormModel, FWFormView, FWFormBrowse com validação de campos e tratamento de eventos
- **Multi-thread:** `StartJob()` (execução em VM isolado, semântica de work process) e `FWGridProcess` (pool de threads com `SetThreadGrid`, `CallExecute`, `StopExecute`, `IsFinished`, meters e log); `advplc check arq1 arq2 ...` verifica N

arquivos em paralelo

- **Renderer web (PO-UI)**: `advplc serve programa.prw` executa o programa no servidor e renderiza a interface no browser com PO-UI (embutido no binário): console e diálogos em tempo real, `FWMBrowse` → `po-table` com dicionário SX3, formulários `po-dynamic-form`, MSDIALOG legado (`@ SAY/GET/BUTTON`) como modal por heurística de grade e hot reload com `--watch`
- **Motor de inferência LLM** (`pkg/llm` + classe `LLM`): carrega modelos GGUF quantizados em I2_S (BitNet/Falcon3-1.58bit) e gera texto direto do AdvPL/TLPP — 100% Go, sem CGO, com kernel SIMD AVX2 em amd64 e fallback escalar em qualquer outra arquitetura
- **Servidor MCP nativo** (`pkg/mcp` + classe `MCPServer`): expõe funções AdvPL/TLPP como "tools" de um servidor MCP real (JSON-RPC 2.0 sobre stdio) — funciona de verdade, validado com o SDK oficial do MCP
- **Servidor REST nativo** (`pkg/rest` + classe `WSRestServer`): sobe um servidor HTTP real (`net/http` puro) e expõe `User Function` anotadas com `@Get` / `@Post` / `@Put` / `@Patch` / `@Delete` como rotas, com path params (`/clientes/{id}`), corpo JSON e dispatch real para a função AdvPL — o DSL clássico `WSRESTFUL` / `WSMETHOD` continua só reconhecido na sintaxe (ver Servidor REST)
- **Núcleo de Tensor (float32)**: classe `Tensor` acelerada em Go (`pkg/tensor`) — `MatMul`, elementwise com broadcast, reduções, ativações, `Softmax`, `Argmax`, `IndexRows` — para construir e rodar modelos float com o AdvPL orquestrando; ver Núcleo de Tensor
- **Autodiff + treino (float32)**: motor de diferenciação reversa (`pkg/autograd`) com a classe `Variable` (tape + `Backward`), ops diferenciáveis (MatMul, Add, Mul, Relu, Sum, Mean, MSE) e otimizador `SGD` — treina modelos float com o AdvPL orquestrando; ver Autodiff e treino
- **Cliente HTTP nativo**: `FWHttpGet` / `FWHttpPost` / `FWHttpPut` / `FWHttpPatch` / `FWHttpDelete` + `FWHttpBody` / `FWHttpStatus` / `FWHttpError` — requisições HTTP com suporte a certificados PKCS#12 (.pfx/.p12), timeout 30s e TLS com verificação; ver Cliente HTTP Nativo
- **DynCall** (classe `tRunDll`): chama funções/métodos de DLL/SO dinamicamente (C e C++, com mangling Itanium real), sem CGO, via `purego`; ver DynCall
- **gRPC embarcado** (classes `GRPCServer` / `tGrpc`): framework gRPC real (`google.golang.org/grpc`) — sobe um servidor gRPC real com Server Reflection a partir de `User Function`, e/ou consome qualquer servidor gRPC como cliente descobrindo o schema em runtime (sem `.proto` gerado em tempo de compilação); ver Servidor/Cliente gRPC
- **Smart Link TOTVS** (classe `FwTotvsLinkClient`): cliente HTTP real com OAuth2 `client_credentials` para o mecanismo de mensageria do Smart Link (TechFin, RH Insights); ver Cliente TOTVS Smart Link
- **Cliente FTP** (classe `TftpClient`): cliente FTP real (RFC 959, modo passivo) — upload/download/listagem/renomear/apagar; ver Cliente FTP
- **Criptografia** (`Argon2id`, classe `tPBKDF2`): Argon2id (RFC9106) e PBKDF2 (SHA1–SHA3-512) via `golang.org/x/crypto`; ver Criptografia
- **Classes utilitárias TLPP** (`tHashMap`, `tJsonParser`, `tUnicode`): formas OOP documentadas pela TDN, interoperando com a API funcional já existente; ver Classes utilitárias TLPP

Servidor MCP (`MCPServer`)

```
User Function McpDemo()
    Local oMCP := MCPServer():New("meu-servidor", "1.0.0")
    oMCP:AddTool("soma", "Soma dois números", ;
        '{"type":"object","properties":{"a":{"type":"number"},"b":{"type":"number"},"required":["a","b"]}',
        "ToolSoma")
    oMCP:Serve() // bloqueia lendo/escrevendo em stdin/stdout
Return

User Function ToolSoma(oArgs)
Return cValToChar(oArgs:A + oArgs:B)
```

Roda com `advplc run meu_programa.prw` normalmente — não precisa de comando novo. O `MCPServer` implementa o protocolo MCP (Model Context Protocol) via JSON-RPC 2.0 sobre stdio — `initialize`, `tools/list`, `tools/call` — em Go puro, sem CGO, sem dependências externas. Cada tool chamada roda a função AdvPL correspondente numa VM isolada (mesmo mecanismo do `StartJob`).

Método	Descrição
<code>New(cNome, cVersao)</code>	Cria o servidor
<code>AddTool(cNome, cDescricao, cSchemaJSON, cNomeFuncao)</code>	Registra uma tool; a função recebe um objeto com os argumentos (<code>oArgs:CAMPO</code>) e retorna o texto do resultado
<code>Serve()</code>	Sobe o loop stdio (bloqueia)

Validado com o SDK oficial em Python do MCP (`cmd/advplc/mcp_integration_test.go`).

Servidor REST (`WSRestServer`)

```
@Get("/clientes/{id}")
User Function GetCliente(oParam)
    Local jRet := JsonObject():New()
    jRet["id"] := oParam:ID // path param populado automaticamente
    jRet["nome"] := "Cliente " + oParam:ID
Return jRet

@Post("/clientes")
User Function NovoCliente(oParam)
    Local jRet := JsonObject():New()
    jRet["criado"] := .T.
    jRet["nome"] := oParam:NOME // campo do corpo JSON da requisição
Return jRet

User Function RestDemo()
    Local oRest := WSRestServer():New("meu-servidor-rest", "1.0.0")
    // @Get/@Post/@Put/@Patch/@Delete acima já viram rota automaticamente;
    // AddRoute cobre o caso de registrar manualmente:
    oRest:AddRoute("GET", "/status", "GetStatus")
    oRest:Serve(8080) // bloqueia servindo HTTP na porta 8080
Return
```

Roda com `advplc run meu_programa.prw` normalmente. O `WSRestServer` sobe um `net/http.Server` real (sem CGO, sem dependências externas) e, ao ser criado, varre todas as `User Function` do programa procurando anotações `@Get / @Post / @Put / @Patch / @Delete("/path")` para registrar como rotas automaticamente — path params (`{id}`) via roteador nativo do Go 1.22+, query string e corpo JSON mesclados num único objeto de argumento (`oParam:CAMPO` , maiúsculo) passado para a função. Cada requisição roda a função numa VM isolada (mesmo mecanismo do `MCPServer / StartJob`), banco e bytecode compartilhados. O retorno da função vira o corpo JSON da resposta (200); erro vira 500; path não registrado vira 404; verbo não registrado num path existente vira 405.

Método	Descrição
<code>New(cNome, cVersao)</code>	Cria o servidor e auto-registra rotas de funções anotadas
<code>AddRoute(cVerbo, cPath, cNomeFuncao)</code>	Registra uma rota manualmente (<code>cPath</code> aceita <code>{param}</code>)
<code>Serve(nPorta)</code>	Sobe o servidor HTTP na porta indicada (bloqueia)

Limitação conhecida: o DSL clássico `WSRESTFUL <nome> ... WSMETHOD <verbo> PATH "... " ... ENDWSRESTFUL` é reconhecido pelo parser mas não executado — o verbo e o `PATH` são descartados ao virar AST, e a implementação do método é ligada a uma instância de classe (não a uma função top-level, que é o que o dispatch HTTP sabe chamar). Para expor esse serviço via HTTP hoje, reescreva no estilo anotações acima ou

registre a rota manualmente com `AddRoute`. Detalhes em `docs/COMPONENT_STATUS.md`.

Testado de ponta a ponta com requisições HTTP reais em `cmd/advplc/rest_integration_test.go`.

Motor de inferência LLM (`LLM`)

```
User Function LlmDemo()
    Local oLLM := LLM():New("/caminho/Falcon3-3B-Instruct-1.58bit/ggml-model-i2_s.gguf")
    ConOut(oLLM:Generate("The capital of France is", 6, 0)) // prompt, nMaxTokens, nTemperatura (0=greedy)
    oLLM:Close()
Return
```

Motor de inferência para modelos **GGUF quantizados em I2_S** (pesos ternários -1/0/+1, formato usado pelo BitNet e por conversões como o Falcon3-3B-Instruct-1.58bit) — escrito inteiramente em Go (`pkg/llm`), sem `llama.cpp`, sem CGO e sem dependências externas. Compila e roda de forma idêntica em Linux, Windows e macOS (amd64/arm64); em amd64 usa um kernel SIMD (AVX2) com detecção de CPU em runtime, caindo automaticamente para um caminho escalar puro em qualquer CPU/arquitetura sem esse suporte.

Métodos da classe `LLM`:

Método	Descrição
<code>New(cCaminhoGGUF)</code>	Carrega o modelo e o tokenizer
<code>Generate(cPrompt, nMaxTokens, nTemperatura)</code>	Gera texto (bloqueia até terminar; <code>nTemperatura<=0</code> = greedy)
<code>Tokenize(cTexto)</code>	Retorna um array de token ids
<code>Decode(aTokens)</code>	Converte token ids de volta em texto
<code>Close()</code>	Libera o modelo

Validado **token a token** contra o `llama.cpp` de referência (ver `pkg/llm/validate_test.go`). Limitações: só arquitetura GGUF `"llama"` com pesos I2_S; sem streaming (ver CHANGELOG para a lista completa).

Renderer web (`advplc serve`)

```
advplc serve tests/mvc_browse_test.prw          # http://localhost:8080
advplc serve programa.prw --port 9000 --watch  # porta própria + hot reload
```

O programa AdvPL/TLPP roda na VM do servidor (mesmo banco `ADVPP.db` de todas as ferramentas) e o browser é o terminal de interface — mesmo modelo do SmartClient HTML do Protheus. O frontend PO-UI/Angular vai embutido no binário (`embed.FS`): nenhuma dependência extra em produção.

- `ConOut` → console em tempo real (SSE)
- `MsgYesNo` / `MsgInfo` /... → diálogos PO-UI que bloqueiam a VM até a resposta
- `FWMBrowse` sobre um alias → `po-table` com colunas/títulos do SX3 e CRUD completo (`po-dynamic-form` gerado do dicionário, soft-delete padrão)
- `DEFINE MSDIALOG` + `@ linha,coluna SAY/GET/BUTTON` → modal PO-UI; os valores digitados voltam para as variáveis do programa
- `--watch`: salvar o fonte recompila e recarrega o browser

Para alterar o frontend: `make web` (requer Node 20+; o resultado compilado é versionado, então `go build` funciona sem Node).

Framework MVC

O compilador AdvPP inclui um framework MVC (Model-View-Controller) completo para construir aplicações estruturadas:

Componentes MVC

FWFormModel - Modelo de dados com definições de campos e validação:

```
oModel := FWFormModel("CustomerModel")
```

FWFormView - View de formulário com componentes e tratamento de eventos:

```
oView := FWFormView("CustomerView", oModel)
```

FWFormBrowse - Componente grid/browse para exibição de dados:

```
oBrowse := FWFormBrowse("CustomerBrowse", oModel)
```

Recursos

- Validação de campos (obrigatório, tamanho, intervalo, personalizado)
- Tratamento de eventos (onChange, onClick, onGotFocus, onLostFocus)
- **Renderização completa de widgets Fyne** (TButton, TGet, TComboBox, TCheckBox, TLabel)
- Estruturas de dados de componentes com renderização visual
- Suporte a diálogos (diálogos, menus, barras de ferramentas, barras de status)
- Eventos de browse (onLineChange, onDbClick, onHeaderClick)

Nota: Componentes MVC renderizam visualmente usando Fyne. Manipuladores de eventos são **parseados e armazenados** no bytecode mas **não conectados à VM** — eventos de clique/foco/seleção do usuário não disparam os callbacks por enquanto. Usar polling ou callbacks manuais para lógica de validação/ação.

Exemplo

```
User Function MVCTest()  
  Local oModel := FWFormModel("CustomerModel")  
  Local oView := FWFormView("CustomerView", oModel)  
  Local oBrowse := FWFormBrowse("CustomerBrowse", oModel)  
  
  // Usar componentes MVC...  
Return .T.
```

Compilação

```
# Compilar todas as ferramentas (advplc, adveditor, advpp-ide)  
make build  
  
# Rodar os testes (build + verificação de todos os fixtures em tests/)  
make test  
  
# Cross-compilar o CLI para Linux/Windows/macOS (amd64 e arm64) em dist/  
make cross  
  
# Gerar pacotes versionados (.tar.gz/.zip) em dist/  
make package VERSION=1.1.0
```

Publicar uma nova versão no GitHub

```
make release VERSION=1.1.0
```

Isso cria e publica a tag `v1.1.0`. O GitHub Actions então compila **nativamente** em Linux, Windows e macOS (incluindo as GUIs Fyne), gera os pacotes (`.tar.gz` , `.zip` , `.deb`) e anexa tudo à Release automaticamente.

Banco de dados compartilhado

Todas as ferramentas (advplc, adveditor, advpp-ide) enxergam o **mesmo** banco SQLite, resolvido nesta ordem:

1. Flag explícita (`advplc run prog.prw --db-path /caminho/banco.db`)
2. Variável de ambiente `ADVPP_DB`
3. Banco configurado em `~/advpp/advpp_config.json` (só se esse arquivo já existir — configurar isso é o que torna o banco "global")
4. Padrão: `./advpp.db` no diretório de trabalho atual — criado automaticamente (`RetSqlName` / `DbSelectArea` /etc. funcionam mesmo sem nenhuma tabela ainda; use o AdvEditor no mesmo diretório para criar tabelas, campos e índices nesse banco)

O driver SQLite é 100% Go (modernc.org/sqlite) — sem CGO, sem dependências externas, idêntico em Linux, Windows e macOS.

Uso

Compilador de Linha de Comando

```
# Executar arquivo fonte AdvPL/TLPP (compila em memória e executa)
./advplc run program.prw

# Compilar fonte para arquivo de bytecode
./advplc compile program.prw -o program.bytecode

# Executar arquivo de bytecode compilado
./advplc exec program.bytecode

# Construir executável standalone (embute bytecode e runtime)
./advplc build program.prw -o program

# Construir como app desktop (obrigatório para programas com MSDIALOG) –
# a janela Fyne abre sempre, sem tentar console primeiro, e no Windows o
# binário é linkado no subsistema GUI (-H=windowsgui: nenhuma janela de
# console aparece, nem por um instante)
./advplc build program.prw -o program --gui

# Verificar apenas sintaxe
./advplc check program.prw

# Imprimir estrutura AST
./advplc ast program.prw

# Imprimir bytecode
./advplc bytecode program.prw
```

IDE Gráfica

```
# Iniciar ambiente de desenvolvimento gráfico
./advpp-ide
```

A IDE gráfica fornece:

- **Editor de Código:** Editor de texto multi-linha com suporte para arquivos `.prw`, `.tlpp` e `.prg`
- **Operações de Arquivo:** Funcionalidades Novo, Abrir, Salvar, Salvar Como

- **Explorador de Projeto:** Navegador de arquivos mostrando diretório atual com destaque de arquivos fonte
- **Integração de Build:** Comandos Compilar, Executar e Compilar & Executar
- **Console de Saída:** Mostra resultados de compilação e saída do programa
- **Suporte a Diálogos:** Funções MsgInfo, MsgStop, MsgAlert e MsgYesNo exibem diálogos Fyne
- **Renderização MVC:** Componentes FWFormModel, FWFormView, FWFormBrowse renderizam visualmente via Fyne (event handling parcial)

Suporte de Linguagem

Recursos AdvPL

- User Function, Static Function, Function declarations
- Escopos de variável Local, Private, Public, Static
- If/Elseif/Else/EndIf, For/Next (inclusive `Step` negativo/descendente), While/EndDo, Do Case/EndCase
- `Loop` (continue) e `Exit` (break) em loops, com aninhamento correto
- Tratamento de erro Begin Sequence/Recover/End Sequence
- Blocos de código `{|| expr }`
- Class/EndClass com Data, Method, Constructor
- Implementação de método fora do bloco de classe
- Acesso a campo de alias `SA1->A1_NOME`
- Auto-referência `::property`
- Todos os tipos de dados AdvPL: Character, Numeric, Logical, Date, Array, Code Block, Nil, Object
- `If()` / `IIF()` com 3 argumentos fazem curto-circuito (avaliam só o ramo escolhido)
- `Private` / `Public` com escopo dinâmico (visíveis às funções chamadas)
- Closures aninhadas: codeblocks capturam Locais N níveis acima por referência

Recursos Adicionais TLPP

- Tipagem estática com palavra-chave `as`
- Tratamento de erro Try/Catch/EndTry
- Declarações de namespace
- Modificadores de acesso (Public, Private, Protected)
- Anotações REST (@Get, @Post, @Put, @Patch, @Delete) - executadas de verdade via `WSRestServer`, ver Servidor REST
- Suporte JSON inline com métodos JsonObject
- Identificadores longos (com namespace)
- Tipos Integer, Double, Decimal, Variant, Variadic
- Parsing de sintaxe WSRESTFUL/WSSERVICE (DSL clássico — reconhecido, execução ainda não suportada; ver limitação em Servidor REST)

Nota: o DSL clássico `WSRESTFUL` / `WSMETHOD` / `ENDWSRESTFUL` é parseado mas não executado (o verbo/PATH são descartados no parser e o dispatch exigiria chamar método de instância). As anotações `@Get` / `@Post` / `@Put` / `@Patch` / `@Delete` sobre `User Function`, por outro lado, sobem um servidor HTTP real via `WSRestServer`.

Cliente HTTP Nativo (FWHttp*)

O AdvPP fornece 8 funções nativas para requisições HTTP com suporte a certificados digitais PKCS#12 (.pfx/.p12). Útil para integração com REST APIs que exigem autenticação TLS mútua (ex.: Emissor Nacional NFS-e, bancos, SEFAZ).

Função	Descrição
<code>FWHttpGet(cUrl, cCert, cSenha)</code>	Requisição GET
<code>FWHttpPost(cUrl, cBody, cType, cCert, cSenha)</code>	Requisição POST
<code>FWHttpPut(cUrl, cBody, cType, cCert, cSenha)</code>	Requisição PUT
<code>FWHttpPatch(cUrl, cBody, cType, cCert, cSenha)</code>	Requisição PATCH

<code>FWHttpDelete(cUrl, cCert, cSenha)</code>	Requisição DELETE
<code>FWHttpBody()</code>	Corpo da última resposta
<code>FWHttpStatus()</code>	Código HTTP da última resposta
<code>FWHttpError()</code>	Mensagem de erro da última requisição

Timeout de 30s por requisição. TLS com verificação de certificado habilitada.
Teste de integração: `cmd/advplc/http_native_test.go` (validado com requisições HTTP reais).

DynCall — chamada dinâmica de DLL/SO (`tRunDll`)

Implementação real do recurso TLPP **DynCall**: carrega uma biblioteca dinâmica (`.dll / .so / .dylib`) em runtime e chama funções C ou métodos C++ dela, com marshaling real de tipos (inclusive `double / float` no registrador correto da ABI) — via `purego`, **sem CGO**, nas 3 plataformas.

```
#include "tlpp-core.th"

Function U_DynCall()
  Local oDll as Object
  Local lOk as Logical

  If (IsSrvUnix())
    oDll := tRunDll():New("./dllc.so")
  Else
    oDll := tRunDll():New("dllc.dll")
  EndIf

  lOk := oDll:callFunction("add", "III", nValue, 4, 8) // retorno lógico – ver nota abaixo
  oDll:Free()
Return
```

Método	Descrição
<code>New(cDllName)</code>	Carrega a DLL/SO; retorna o objeto <code>tRunDll</code>
<code>Free()</code>	Descarrega a DLL — lógico
<code>CallFunction(cFunc, cSignature, xRet, ...)</code>	Chama função C (<code>extern "C"</code> , sem name mangling) — lógico
<code>CallMethod(cMetodo, cSignature, xRet, ...)</code>	Chama método de DLL C++ (<code>"Classe::metodo(tipos)"</code> , resolvido via mangling Itanium — GCC/Clang) — lógico
<code>GetVar(cNome, cSignature, xRet)</code> / <code>SetVar(cNome, cSignature, xValor)</code>	Lê/escreve variável global exportada pela DLL — lógico
<code>NewPointer()</code> / <code>NewObj([nBytes])</code> / <code>FreeObj(oObj)</code>	Handles opacos de ponteiro/objeto C++; <code>NewObj(nBytes)</code> aloca memória real (fixada com <code>runtime.Pinner</code>) para servir de <code>this</code> a um construtor chamado via <code>CallMethod</code>
<code>StrLen(nRet, oPointer)</code>	Varre até <code>\0</code> ; lógico (<code>nRet</code> não é escrito, limitação de <code>@var</code>)
<code>StrCpy(oPointer, nMaxSize)</code> / <code>MemCpy(oPointer, nBytes)</code>	Lê o buffer C e retorna a String lida diretamente (desvio deliberado do TDN, que documenta saída por referência inaplicável neste VM)
<code>GetTimeout()</code> / <code>SetTimeout(nSeg)</code>	Timeout nominal (default 60s); aceito e armazenado, sem efeito real — a chamada é síncrona in-process

`GetLastError()` / `GetErrorMsg()`

Código/mensagem do último erro

Limitações importantes (documentadas em detalhe em `docs/tdn-known-limitations.md`):

todo método acima documenta retorno lógico na TDN — o valor de fato é sempre saída por referência (`xRet`), que este VM não escreve de volta (mesma limitação arquitetural de `@var` que afeta várias outras categorias deste compilador), exceto quando `xRet` é um objeto `TRunDLLPointer` (mutado in-place de verdade). Passagem de parâmetro por referência via `@` (`int*` / `int&` do lado C/C++) não é suportada — o compilador AdvPP descarta o `@` em qualquer posição de chamada, então use a letra `'P'` explícita + um `TRunDLLPointer` real nesses casos. `CallMethod` só resolve mangling Itanium (GCC/Clang/MinGW) — DLLs compiladas com MSVC não são suportadas. Testado ponta a ponta contra bibliotecas C e C++ reais compiladas por `gcc` / `g++` em tempo de teste (`pkg/vm/dyncall_native_test.go`).

Servidor gRPC (`GRPCServer`) e Cliente gRPC (`tGrpc`)

Implementação real do framework gRPC (`google.golang.org/grpc`)

- `google.golang.org/protobuf`, as libs oficiais) embarcada no compilador — tanto para consumir quanto para expor serviços gRPC, com HTTP/2 e protobuf reais na wire (não simulados).

```
User Function Ping(oParams)
    Local oResp := JsonObject():New()
    oResp["pong"] := "hello " + oParams["name"]
Return oResp

User Function StartGrpcServer()
    Local oServer := GRPCServer():New()
    oServer.AddMethod("Echo", "Ping", "Ping") // serviço, método, função AdvPL
    oServer.Serve(50051) // bloqueia servindo gRPC na porta 50051
Return
```

AdvPL/TLPP não tem um DSL para declarar tipos `.proto` estaticamente — por isso, mesmo espírito de `WSRestServer` / `MCPServer` (que serializam parâmetros/retorno como JSON), `GRPCServer` define em runtime um único tipo de mensagem genérico (`message JsonEnvelope { string json = 1; }`) reaproveitado por toda RPC registrada, com **Server Reflection** **habilitada** — qualquer cliente gRPC real (`grpcurl`, um cliente gerado por `protoc`, ou o próprio `tGrpc` deste compilador) descobre o serviço em runtime e chama sem precisar de arquivo `.proto` nenhum. Os parâmetros chegam na função AdvPL como um `JsonObject` real (acesso por colchete, `oParams["campo"]`, case-sensitive — semântica JSON), e o retorno da função vira o corpo JSON da resposta.

Método (<code>GRPCServer</code>)	Descrição
<code>New()</code>	Cria o servidor
<code>AddMethod(cServico, cMetodo, cFuncao)</code>	Registra uma User Function como handler unário — deve ser chamado antes de <code>Serve()</code>
<code>Serve([nPorta])</code>	Sobe o servidor gRPC (default <code>:50051</code>), bloqueia até <code>Shutdown()</code>
<code>Shutdown()</code>	Encerra graciosamente (<code>GracefulStop</code>), chamada de outra goroutine/job

```
User Function GrpcClientDemo()
    Local oClient := tGrpc():New("smartlink.proto", "localhost", 50051)
```

```

If oClient:isRunning() // conexão HTTP/2 real, connectivity.State real
  oClient:MsgContent := '{"name":"AdvPP"}'
  If oClient:sendMessage() // descobre o método via Server Reflection e invoca
    ConOut(oClient:waitForMessages())
  EndIf
EndIf
Return

```

`tGrpc` é o cliente correspondente: abre uma conexão HTTP/2 real (`grpc.NewClient`), descobre em runtime — via gRPC Server Reflection, sem precisar de nenhum `.proto` local — quais serviços/métodos o servidor alvo expõe, e invoca dinamicamente (`dynamicpb`) o método cujo nome mais se aproxima do documentado pela TDN (`ClientSetup`, `TenantSetup`, `TenantUndo`, `SendMessage`, `WaitForMessages`, `AckMessage`), populando os campos por casamento de nome contra as propriedades (`ClientInfoProp`, `MsgId`, `MsgType`, `MsgContent`, `MsgAud`, `MsgDeliveryTag`, `MsgAckDeliveryTag`, `MsgAck`). Testado ponta a ponta (cliente ↔ servidor reais, inclusive `tGrpc` ↔ `GRPCServer` entre si) — ver `docs/tdn-known-limitations.md` para o detalhe completo de cada método e a limitação real que resta (sem o `.proto` proprietário do Smart Link/TOTVS, os nomes usados são os mais prováveis, não confirmados contra o serviço TOTVS real).

Cliente TOTVS Smart Link (`FwTotvsLinkClient`)

Implementação real (HTTP, `net/http`) da classe TDN `FwTotvsLinkClient` — o mecanismo de troca de mensagens do Smart Link (fila via HTTP, autenticação OAuth2 `client_credentials`) usado por aplicações TOTVS reais (TechFin, RH Insights) para se comunicar com produtos TotvsApps.

```

User Function SmartLinkDemo()
  Local oClient := FwTotvsLinkClient():New(.T.) // .T. = refresh do token
  Local lOk      := oClient:SendAudience("PedidoCriado", "Antecipa", '{"valor":100}')
  If lOk .and. oClient:Receive()
    ConOut(oClient:GetMessage():toJson())
    oClient:Success()
  Else
    ConOut(oClient:GetError())
  EndIf
  oClient:Destroy()
Return

```

Método	Descrição
<code>New([lRefreshToken])</code>	Cria o cliente; se <code>.T.</code> , já busca um token OAuth2
<code>Send(cType, cMessage)</code> / <code>SendAudience(cType, cAudience, cMessage, [aHeader])</code>	Envia mensagem (POST real)
<code>Receive()</code>	Consulta a fila (GET real); <code>.F.</code> quando vazia
<code>GetMessage()</code>	Objeto JSON real da última mensagem recebida
<code>Success()</code> / <code>Fail()</code>	Confirma processamento / envia para a fila DLQ
<code>GetTenantClient()</code>	Tenant configurado
<code>GetError()</code>	Última mensagem de erro

`Set/GetRefreshToken()`, `Destroy()`

Configuração e limpeza

Endpoint, credenciais OAuth2 e tenant são lidos de variáveis de ambiente (`ADVPP_SMARTLINK_BASEURL`, `ADVPP_SMARTLINK_TOKENURL`, `ADVPP_SMARTLINK_CLIENTID`, `ADVPP_SMARTLINK_CLIENTSECRET`, `ADVPP_SMARTLINK_TENANTID`) — sem configurar, as chamadas falham honestamente (erro real de conexão, nunca um sucesso simulado). Detalhe completo (paths REST, formato de `GetMessage()`, limitações) em `docs/tdn-known-limitations.md`.

Cliente FTP (`TFtpClient`)

Cliente FTP real (RFC 959) via `net` / `net/textproto`, com modo passivo (PASV) para listagem e transferência de arquivos.

```
User Function FtpDemo()
    Local oFtp := tFtpClient().New()
    Local nRet := oFtp:FTPConnect("ftp.exemplo.com", 21, "usuario", "senha")
    If nRet == 0
        oFtp:SendFile("local.txt", "remoto.txt")
        oFtp:ReceiveFile("remoto.txt", "baixado.txt")
        oFtp:Close()
    EndIf
    Return
```

`New`, `FTPConnect`, `Directory`, `ChDir` / `CdUp` / `Mkdir` / `Rmdir`, `SendFile` / `ReceiveFile`, `RenameFile` / `DeleteFile`, `GetType` / `SetType`, `NoOp`, `Quote`, `GetHelp`, `GetMLCount` / `GetMLLine`, `GetLastResponse` / `GetCurDir`, `Close` — testado ponta a ponta (upload/download com round-trip de conteúdo verificado) contra um servidor FTP real.

Criptografia — Argon2id e PBKDF2 (`tPBKDF2`)

```
User Function CryptoDemo()
    Local cHash := Argon2id("minhaSenha", "meuSalt") // RFC9106, defaults TDN
    Local oPBKDF2 := tPBKDF2().New()
    oPBKDF2:setPassword("minhaSenha")
    oPBKDF2:setSalt("meuSalt")
    oPBKDF2:setDigest("SHA256")
    oPBKDF2:encrypt()
    ConOut(oPBKDF2:getKeyHex())
    Return
```

`Argon2id(cText, cSalt, [nMemoryCost], [nIterations], [nThreads], [nHashLen], [nLanes])` via `golang.org/x/crypto/argon2`. `tPBKDF2` (`New`, `setPassword` / `setSalt` / `setIteration` / `setKeylength` / `setDigest`, `encrypt`, `getKeyHex` / `getKeyRaw` / `getKeyBase64` / `getKeyUrl_Base64`, `release`, `getLastError`) via `golang.org/x/crypto/pbkdf2` — SHA1 a SHA3-512.

Classes utilitárias TLPP (`tHashMap`, `tJsonParser`, `tUnicode`)

```
User Function UtilDemo()
    Local oHash := tHashMap().New()
    oHash:Set("chave", 42)

    Local oJson := tJsonParser().New()
    Local aCampos := {}, nLido := 0, oMapa := tHashMap().New()
```

```
oJson:Json_Hash({'a':1}', 6, aCampos, nLido, oMapa)

Local oUni := tUnicode():New()
ConOut(oUni:ConvertEncoding("teste", "cp1252", "UTF-8"))
Return
```

`tHashMap` (`New` / `Set` / `Get` / `Del` / `List` / `Clean` / `nStatus`) é a forma OOP da classe documentada pela TDN — interopera com a API funcional histórica (`HMNew` / `HMSet` / `HMGet` /...). `tJsonParser` (`New` / `Json_Hash` / `Json_Parser` / `json_ok`) e `tUnicode` (`New` / `Normalize` / `ConvertEncoding`), formas normais NFC/NFD/NFKC/NFKD via golang.org/x/text/unicode/norm) completam a família de classes "TLPP - Classes úteis" da TDN.

Funções de I/O, arquivo e sistema

O runtime expõe I/O de disco, uma API de handle de arquivo para streaming e uma chamada de sistema — todas com semântica AdvPL nativa, em Go puro (sem CGO).

I/O de disco (arquivo inteiro)

Função	Descrição
<code>MemoRead(cArq)</code>	Lê o arquivo inteiro e retorna como string (<code>""</code> se não existir)
<code>MemoWrite(cArq, cTexto)</code>	Grava a string no arquivo; retorna <code>.T.</code> em sucesso (alias: <code>MemoWrit</code>)
<code>FErase(cArq)</code>	Apaga o arquivo; <code>0</code> em sucesso, <code>-1</code> em erro

Console interativo

Função	Descrição
<code>ConIn([cPrompt])</code>	Lê uma linha do stdin (sem o <code>\n</code>); <code>""</code> no EOF. Contraparte de <code>ConOut</code> para programas de console interativos (REPL, chat)

Handle de arquivo (streaming — arquivos grandes)

Função	Descrição
<code>FCreate(cArq[, nAttr])</code>	Cria/trunca o arquivo; retorna handle (<code>>=1</code>) ou <code>-1</code>
<code>FOpen(cArq[, nMode])</code>	Abre existente; bit <code>0</code> de <code>nMode</code> = escrita (<code>0</code> = leitura). Handle ou <code>-1</code>
<code>FReadStr(nH, nBytes)</code>	Lê até <code>nBytes</code> e retorna string (<code>""</code> no fim do arquivo)
<code>FWrite(nH, cBuffer[, nBytes])</code>	Grava; retorna nº de bytes escritos
<code>FSeek(nH, nOffset[, nOrigin])</code>	<code>0</code> =início, <code>1</code> =atual, <code>2</code> =fim; retorna a nova posição
<code>FClose(nH)</code>	Fecha o handle; <code>.T.</code> / <code>.F.</code>
<code>FError()</code>	Código do último erro de I/O (<code>0</code> = sem erro)

A leitura usa `FReadStr` (retorna a string lida) em vez do `FRead` com buffer por referência — os natives da VM recebem valores, não lvalues, então byref em uma `Local` string não propagaria. `FReadStr` é a forma AdvPL genuína para isso.

```
// Streaming de um arquivo grande em blocos de 4 KB
Local nH := FOpen("dados.txt", 0)
```

```
Local cBloco := FReadStr(nH, 4096)
While Len(cBloco) > 0
  // ... processa cBloco ...
  cBloco := FReadStr(nH, 4096)
End
FClose(nH)
```

Chamada de sistema

Função	Descrição
<code>WaitRun(cCmd)</code>	Executa <code>cCmd</code> no shell do SO (cross-platform <code>sh -c</code> / <code>cmd /c</code>), herda stdio, espera e retorna o <i>exit code</i> (<code>0</code> = sucesso)

Para **capturar** a saída de um comando, use o padrão AdvPL de redirecionar para arquivo e ler — com a API de handle isso funciona para saída arbitrariamente grande, em streaming:

```
WaitRun("gerar_relatorio.sh > saida.txt")
Local nH := FOpen("saida.txt", 0)
Local cSaida := FReadStr(nH, 65536) // ou em blocos, para arquivos enormes
FClose(nH)
```

Álgebra linear ternária (BLAS)

Função	Descrição
<code>MatVecTern(aMat, aVecTern)</code>	Produto matriz-vetor <i>multiply-free</i> onde o vetor é ternário (<code>-1 / 0 / +1</code>): <code>result[i] = Σ_j sign(vec[j])·mat[i][j]</code> — só soma/subtração, o kernel do BitNet. <code>aMat</code> é um array de M linhas (cada uma um array de N números); <code>aVecTern</code> tem N entradas

Base para redes neurais ternárias em AdvPL: peso/ativação em `{-1,0,+1}` eliminam a multiplicação, viabilizando treino e inferência sem BLAS de ponto flutuante nem GPU (ver `tests/llm/pt_nn.prw`).

Funções de array de ordem superior (com bloco de código)

Honram um bloco de código `{[...]| ... }` de verdade (avaliado pela VM):

Função	Descrição
<code>ASort(aArr, [nIni], [nQtd], [bOrder])</code>	Ordena in-place; <code>bOrder(x,y)</code> retorna <code>.T.</code> se <code>x</code> vem antes de <code>y</code> (sem bloco: ascendente)
<code>AEval(aArr, bBloco, [nIni], [nQtd])</code>	Aplica <code>bBloco(elem, i)</code> a cada elemento
<code>AScan(aArr, uVal bBloco, [nIni], [nQtd])</code>	Posição do 1º elemento igual a <code>uVal</code> ou onde <code>bBloco(elem)</code> é <code>.T.</code> ; <code>0</code> se não achar
<code>File(cArq)</code>	<code>.T.</code> se o arquivo existe (não-diretório)
<code>GetNames(oJson)</code>	Array com as chaves de um JsonObject, na ordem de inserção

Os blocos são **closures de verdade**: capturam Locais do escopo envolvente por referência — leitura e escrita — inclusive quando o bloco escapa da função que o criou (estado persistente). Ex.: `AEval(a, {|x| nSoma := nSoma + x})` acumula no `nSoma` externo; `{|| nN := nN + 1}` retornado por uma função vira um contador com estado próprio. Captura em profundidade funciona completamente — bloco-dentro-de-bloco captura Locais N níveis acima por referência.

Núcleo de Tensor

A classe `Tensor` (`float32`) guarda os dados como `[]float32` plano em Go — fora da representação *boxed* de `Value` — e roda kernels de forward em Go puro. O AdvPL orquestra; o Go faz a conta.

```
Local oX := Tensor():FromArray({1,2}, {1,2})
Local oW := Tensor():Rand({2,3}, 0.1)
Local oH := oX:MatMul(oW):Relu()           // [1,3]
Local oY := oH:Softmax(2)                  // softmax por linha
Local nId := oY:Argmax()                   // classe prevista (1-based)
```

Construtores: `New(aForma)`, `FromArray(aDados, aForma)`, `Rand(aForma, nEscala)`.
Métodos: `Shape`, `Size`, `Get / Set`, `ToArray`; `Add / Sub / Mul / Div` (com broadcast de escalar e linha/coluna), `AddScalar / MulScalar`; `MatMul`, `Transpose`, `Reshape`; `Sum / Mean / Max / Argmax` (sem eixo → número; com eixo → Tensor); `Exp / Log / Sqrt / Relu / Tanh / Sigmoid / Gelu`; `Softmax`; `IndexRows` (lookup de embedding). Erros de forma são capturáveis por `Try/Catch`.

Precisão selecionável (float32 / float64)

O dtype é escolhível **por tensor**: `float32` é o default (rápido, usado pelo ML) e `float64` entra sob demanda para cálculo que exige exatidão (base do kernel de álgebra linear/geometria). A precisão escalar do AdvPL já é float64; isto leva a dupla precisão ao kernel de Tensor.

```
Local oA := Tensor():New({2,2}, "float64")           // dtype float64
Local oB := Tensor():FromArray({1,2,3,4}, {2,2}, "float64")
ConOut(oB:DType())                                   // "float64"
Local oC := oA:ToFloat64()                           // converte f32 -> f64
ConOut(oB:Dot(oB))                                    // produto interno
ConOut(Tensor():FromArray({3,4},{2},"float64"):Norm()) // norma L2 = 5
```

Métodos de dtype: `DType()` (`"float32" / "float64"`), `ToFloat32()` / `ToFloat64()`, `Dot(oOutro)` (produto interno) e `Norm()` (L2). As ops (`Add / MatMul / ...`) respeitam o dtype e **promovem a float64** se qualquer operando for f64; o caminho float32 permanece idêntico (o ML não é afetado). Propagação de f64 pelo autodiff fica para um ciclo futuro (álgebra/geometria não usam gradiente).

Este ciclo entrega o **forward** (inferência) + precisão dupla. Autodiff/treino veio em ciclos seguintes.

Álgebra linear (float64)

Sobre o Tensor float64, operações de álgebra linear em Go puro (não-diferenciáveis — cálculo, não treino):

```
Local oA := Tensor():FromArray({4,7,2,6}, {2,2}, "float64")
ConOut(oA:Det())           // determinante
Local oX := oA:Solve(oB)    // resolve A·x = b (b vetor [n] ou [n,k])
Local oInv := oA:Inv()       // inversa (A·Inv ≈ I)
Local aQR := oA:QR()        // {Q, R} — Householder (Q·R ≈ A)
Local aEig := oS:EigSym()    // {valores[n], vetores[n,n]} de matriz simétrica (Jacobi)
```

- `Det()` determinante via LU (pivô parcial); singular → 0.
- `Solve(oB)` resolve $A \cdot x = b$ por substituição direta/reversa sobre a LU.
- `Inv()` inversa resolvendo $A \cdot X = I$; singular → erro capturável.
- `QR()` → `{Q, R}` por refletores de Householder (`Q` ortogonal, `R` triangular sup.).
- `EigSym()` → `{valores, vetores}` de matriz **simétrica** por rotações de Jacobi (autovalores decrescentes; colunas de `vetores` = autovetores). Não-simétrica → erro.
- `SVD()` → `{U, S, V}` (decomposição em valores singulares, Jacobi de um lado;

$A \approx U \cdot \text{diag}(S) \cdot V^T$, S decrescente, suporta retangular $m \times n$).

- **Eig()** → {reais, imag} — todos os autovalores de matriz **não-simétrica** (real), incluindo **pares complexos conjugados**, via redução a Hessenberg + QR de duplo shift (Francis/hqr). Para autovalor complexo, **imag** traz o par $\pm$.

Erros (não-quadrada, singular, não-simétrica em **EigSym**, dims incompatíveis) são **ErrorValue** capturáveis.

Geometria espacial

Funções nativas sobre vetores/pontos como arrays (**{x,y}** / **{x,y,z}**), em float64:

```
ConOut(VecCross({1,0,0}, {0,1,0})) // produto vetorial 3D -> {0,0,1}
ConOut(VecDot({1,2,3}, {4,5,6}))   // produto escalar
ConOut(VecNorm({3,4}))              // magnitude -> 5
ConOut(VecDist({0,0}, {3,4}))       // distância euclidiana -> 5
ConOut(VecAngle({1,0}, {0,1}))      // ângulo (rad) -> π/2
Local aU := VecNormalize({3,4})     // vetor unitário
Local aR := RotateVec2({1,0}, nTheta) // rotação 2D
Local aP := RotateVec3({1,0,0}, "z", nTheta) // rotação 3D em torno de x/y/z
```

Também **VecAdd**, **VecSub**, **VecScale**. Erros (dims incompatíveis, cross fora de 3D) são capturáveis.

Aritmética e estatística

Funções escalares adicionais: **Atan2(y,x)**, **Log10(x)**, **Pow(b,e)**, **Ceil(x)**, **Sign(x)**, **Sinh/Cosh/Tanh(x)**, **Gcd(a,b)**, **Lcm(a,b)**, **Fact(n)**.

Estatística sobre arrays: **Mean(a)**, **Variance(a)**, **StdDev(a)** (amostrais), **Median(a)**, **LinReg(aX, aY)** → {a, b} de $y = a + b \cdot x$ (mínimos quadrados), **Interp(aX, aY, x)** (interpolação linear).

Autodiff e treino

Sobre o núcleo de Tensor, a classe **Variable** grava um tape de operações e **Backward()** propaga gradientes (reverse-mode autodiff). Otimizadores **SGD** e **Adam** permitem treinar modelos float — o AdvPL orquestra o laço; o Go faz forward e backward.

```
Local oW := Variable():FromArray(aPesos, {nIn, nOut})
Local oB := Variable():FromArray(aBias, {nOut})
Local oOpt := Adam():New({oW, oB}, 0.001) // Adam optimizer (v1.9.0+)
// laço de treino:
Local oPred := oX:MatMul(oW):Add(oB):Relu()
Local oLoss := oPred:MSE(oY)
oOpt:ZeroGrad()
oLoss:Backward() // preenche oW:Grad(), oB:Grad()
oOpt:Step()      // oW := oW - lr*grad
```

Ops diferenciáveis: **MatMul**, **Add** (com broadcast), **Mul**, **Relu**, **Sum**, **Mean**, **MSE**, **Tanh**, **Sigmoid**, **Gelu**, **IndexRows** (embedding). **oV:Value()** / **oV:Grad()** devolvem o **Tensor** de valor/gradiente. Corretude validada por verificação numérica de gradiente (diferenças finitas) no **go test**.

Loss de classificação: **oLoss := oLogits:SoftmaxCE(aAlvo)** (softmax + cross-entropy, alvo por índices de classe — adicionado v1.9.0+).

Módulos: **Linear():New(nIn, nOut)** e **Embedding():New(nVocab, nDim)** encapsulam parâmetros + **Forward**; **oMod:Params()** devolve os pesos para o otimizador; **Fit(bPasso, nEpocas)** roda o laço de treino avaliando um codeblock por época (adicionado v1.9.0+).

Exemplos de IA em AdvPL puro

Entregue (**tests/llm/**). Modelos escritos inteiramente em AdvPL: Markov chains, retrieval-based Q&A, ternary neural networks e LM treinado por gradiente — todos rodando de verdade sobre o motor (LLM, Tensor, autodiff)

e os frameworks de treino deste compilador, não prototipados em Go.

LM Neural Treinado por Gradiente

Entregue (`tests/llm/pt_neural.prw`). LM neural char-level treinado de verdade por descida de gradiente, montado 100% sobre o stack de ML do AdvPP (Tensor + autodiff/treino). Arquitetura NPLM (Bengio 2003):

```
oEmb := Embedding():New(V, D)           // tabela de embeddings [V, D]
oL1  := Linear():New(k*D, H)
oL2  := Linear():New(H, V)
// forward: contexto de k chars -> próximo char
oLog := oL2:Forward( oL1:Forward( oEmb:Forward(aX):Reshape({N, k*D}) ):Tanh() )
oLoss := oLog:SoftmaxCE(aAlvo)           // perda
// treino: Adam sobre Params() dos 3 módulos, via Fit(bPasso, nEpcas)
```

Demonstra ponta a ponta: tokenizar → treinar → gerar, provando que o stack float treina um LM de verdade.

LM de Código AdvPL Orientado a Desenvolvimento

Entregue (`tests/llm/dev_nn.prw`). LM treinado em token-level (não char-level) sobre código AdvPL do repositório (corpus gerado por `tests/llm/build_corpus.sh`). Demonstra como o stack de ML AdvPP pode ser usado para gerar/completar código a partir de um prefixo, com REPL de autocomplete.

Limitações Conhecidas

REST API: Anotações vs. DSL Clássico

Status: Anotações `@Get / @Post / @Put / @Patch / @Delete` sobre `User Function` são **totalmente suportadas** com dispatch HTTP real. O DSL clássico `WSRESTFUL <nome> ... WSMETHOD <verbo> PATH "..." ... ENDWSRESTFUL` é **apenas parseado** — reconhecido sintaticamente, mas não executado.

Motivo: O DSL clássico liga a implementação do método a uma instância de classe (`WSSERVICE`), enquanto o dispatch HTTP via `WSRestServer:Serve()` só pode chamar funções top-level. Reescrever para ambos exigiria cirurgia de parser + mecanismo de dispatch novo sem caso de uso nos corpora validados.

Solução: Use anotações ou registre manualmente com `AddRoute` :

```
@Get("/clientes/{id}")
User Function GetCliente(oParam)
// ...
Return oResult

User Function RestDemo()
  Local oRest := WSRestServer():New("api", "1.0")
  // Anotação acima auto-registra; ou registre manualmente:
  oRest:AddRoute("GET", "/status", "GetStatus")
  oRest:Serve(8080)
Return
```

Locks de Registro (RecLock/MsUnlock)

Status: `RecLock()` e `MsUnlock()` implementam **semáforos por tabela** desde a v2.0.3 — **não são mais no-ops**. Cada alias tem um semáforo exclusivo que garante acesso serial ao mesmo registro em `WaitRecno()` .

Limitação: Semáforos são **intra-VM** — protegem threads dentro de um único processo `advplc` . Aplicações multi-processo (ex.: múltiplas instâncias de `advplc` num servidor de apps) **não** veem os locks um do outro — o banco SQLite usa `busy_timeout` e WAL mode para evitar contenção, mas sem consenso distribuído, a concorrência inter-processo é arriscada para operações ACID fortes.

Recomendação: Para produção multi-processo, implemente locks no próprio banco (ex.: uma tabela de semáforos). **Não** use `GetMV / GetNewPar` para isso — no AdvPP são stubs que apenas ecoam o valor default recebido como argumento, sem consultar nenhuma tabela de parâmetros (SX6); não existe leitura real de configuração nem `SuperGetMV / Pergunte` .

Modo Console Interativo (CLI) vs. App Desktop (`--gui`)

Status: `advplc build programa.prw` sobe um executável com **interface interativa de verdade** se detectar operações de terminal (`FWGetText` , `FWMenuSelect` , `ConIn` , formulários `MSDIALOG` , `FWMBrowse`).

Detecção: Varre o bytecode em busca de `OP_CALL_NATIVE` a UI natives (`FWGetText` , `FWMenuSelect`) e `OP_NEW_INSTANCE` de componentes (`FWMBrowse` , `MSDIALOG` e afins) ou anotações REST (`@Get` / `@Post` /etc). Se nada disso for encontrado (ex.: programa que só usa `ConOut` / `ConIn`), o binário roda sempre em console, mesmo sem TTY — não há UI para mostrar. Se algo for encontrado e `stdin` **não** for um terminal real (TTY) — executável aberto por duplo-clique num gerenciador de arquivos, por exemplo — ele abre uma **janela Fyne de verdade**; não é o mesmo renderer PO-UI/web do `advplc serve`, standalone e web são dois caminhos de UI independentes, e o binário `build` nunca usa PO-UI. Com TTY presente e algo detectado, roda em console.

Três formas de decidir esse comportamento, do mais fraco ao mais forte:

1. **Automático (padrão):** console se TTY ou nada de UI detectado no bytecode; Fyne caso contrário — descrito acima.
2. `ADVPP_FORCE_GUI=1 ./programa` (variável de ambiente, decidida em tempo de **execução**): força a janela Fyne mesmo rodando de um terminal — útil quando um script wrapper quer isso por plataforma, sem recompilar.
3. `advplc build programa.prw -o programa --gui` (flag no **build**, decidida em tempo de compilação): marca o programa como app desktop de forma permanente — a fallback de console nunca é tentada, **e** no Windows o binário é linkado no subsistema GUI (`-H=windowsgui`), então nenhuma janela de console aparece, nem por um instante (sem `--gui`, um `.exe` Windows fica no subsistema console mesmo abrindo uma janela Fyne por cima). **Obrigatório para qualquer programa que use `MSDIALOG`**.

Use `--gui` quando o app é desktop por natureza (e-Gov, GesCon) e será distribuído para usuários finais — é a opção correta para produção no Windows. Reserve `ADVPP_FORCE_GUI` para alternar o comportamento de um binário já compilado sem GUI fixa, ou para testes locais.

Exemplo:

```
advplc build meu_app.prw -o meu_app
./meu_app                                # Console se stdin for TTY, senão Fyne

ADVPP_FORCE_GUI=1 ./meu_app              # Força janela Fyne neste binário, em runtime

advplc build meu_app.prw -o meu_app --gui # GUI fixa no binário (build-time);
./meu_app                                # no Windows, sem subsistema console
```

Motor LLM: Limitações de Modelo

Status: Classe `LLM` carrega modelos GGUF quantizados **apenas em I2_S** (ternário: -1/0/+1, estilo BitNet/Falcon3-1.58bit).

Limitações:

- Quantização: Só **I2_S** suportado; F16/F32 não funcionam
- Streaming: Não há suporte a streaming de token; `Generate()` **bloqueia** até terminar
- Tokenizer: Pré-built na .gguf; não há suporte a tokenizers dinâmicos
- Modelos: Arquitetura deve ser `llama` com pesos I2_S — outras arquiteturas (Qwen, Mistral, etc.) e quantizações (Q4_K, Q6_K) causam erro

Alternativa: Para F16/F32 ou streaming, use uma API externa (ex.: Ollama local com `FWHttpPost`).

Tensor: Precisão Float32 vs Float64

Status: `Tensor` suporta **float32 (default)** e **float64 (optativo)** por instância.

Padrão: Float32 (rápido, para ML) é o tipo padrão. Float64 entra sob demanda para cálculo exato (álgebra linear, geometria). As operações respeitam dtype e promovem para float64 se qualquer operando for f64.

Exemplo:

```
Local oA := Tensor():New({2,2}, "float64")
Local oB := oA:Add(Tensor():New({2,2})) // promoção: resultado é f64
```

Autodiff + Treino: Forward + Backward, SGD e Adam

Status: Classe `Variable` implementa **forward pass + reverse-mode autodiff**, com otimizadores `SGD` e `Adam` (v1.9.0+) e `SoftmaxCE()` diferenciável (loss + gradiente `(softmax - onehot)/N`) para perda de classificação.

Limitações:

- Módulos: Suporte básico a `Linear` e `Embedding`; não há Convolução, RNN, Transformer
- Batch: Forward/backward por exemplo; batches são do usuário (loop manual)

Roadmap: módulos complexos (Convolução, RNN, Transformer) ficam para ciclos futuros.

Treinamento Neural (pt_neural.prw): Pequeno Modelo

Status: LM neural char-level treinado 100% com Autodiff/SGD em AdvPL puro (`pt_neural.prw`).

Limitações:

- **Tamanho:** Modelo pequeno (embedding dim 50, hidden 200); em corpus pequeno pode overfitar
- **Coerência:** Em corpus complexo (e.g., *Dom Casmurro* em prosa literária), gera morfologia correta mas coerência limitada — n-grama local não captura dependências longas
- **Velocidade:** VM interpretada + float em AdvPL = lento; treino de 1 época em corpus 72k tokens leva ~30–90s
- **Sem generalização:** Modelo pequeno em corpus finito tende a decorar + aprender padrão; não generaliza a problemas novos

Ressalva: Prova de conceito que treino real funciona; não é um LLM produção.

Tratamento de Eventos em MVC

Status: `FWFormModel`, `FWFormView`, `FWFormBrowse` definem **manipuladores de eventos** (onChange, onClick, onGotFocus, onLostFocus, onLineChange, onDbClick, onHeaderClick).

Limitação: Manipuladores são **parseados e armazenados** no bytecode, mas **não conectados à VM** — eventos de clique/foco/seleção do usuário não disparam os callbacks.

Impacto: Aplicações MVC renderizam visualmente (Fyne) e aceitam input de usuário (formulários, grids), mas lógica de validação/ação de evento deve ficar no `StartJob` que roda a UI (polling ou callback manual).

Status: △ Parcialmente implementado; execução de evento vem num ciclo futuro.

HTTP: Timeouts e Limites

Status: Classe `FWHttp*` e cliente HTTP nativo implementam:

- Timeout: **30 segundos** por requisição (bloqueante)
- Redirecionamentos: Máximo **5 redirects** (evita loops)
- TLS: Verificação de certificado **habilitada** (erro em cert inválido, self-signed não é aceito sem config extra)

Implicação: URLs com redirect chain >5 falham; serviços com cert self-signed requerem workaround (desabilitar verificação explicitamente via variável de ambiente ou config).

Servidor Standalone: Detecção Console-vs-GUI

Status: Binários standalone (`advplc build`) detectam automaticamente se devem usar console (TTY) ou GUI (Fyne) — ver detalhamento completo, incluindo a flag `--gui` de build-time e `ADVPP_FORCE_GUI` de runtime, em [Modo Console Interativo \(CLI\)](#) vs. [App Desktop](#).

Heurística: Varre bytecode em busca de UI natives (`FWGetText`, `FWMenuSelect`), instanciação de classes de UI (`FWMBrowse`, `MSDIALOG` e afins) e anotações REST (`@Get` / `@Post` /etc). Só sobe GUI Fyne se **algo disso foi detectado E stdin não é um TTY**; caso contrário (nada detectado, ou TTY presente), roda em console — a menos que `--gui` (build-time) ou `ADVPP_FORCE_GUI` (runtime) force Fyne mesmo assim.

Caso de borda: Apps que usam apenas `ConOut` / `ConIn` (sem diálogos) rodam em console mesmo com stdin desacoplado (ex.: em cron ou daemon) — nenhuma interação ocorre, só logs. Use `ADVPP_FORCE_GUI=1` (ou compile com `--gui`) se quiser GUI mesmo sem UI natives detectadas.

Contagem de Funções Nativas

Real: centenas de funções nativas, registradas em dezenas de arquivos

`pkg/vm/*_native.go` (729 registradas na contagem do `CHANGELOG.md [3.0.0]`, crescendo a cada release — não é um número fixo). Muitas são

stubs/no-ops (ex.: `FWCLEARHLP`, `HELP`, `MSDOCUMENT`) por design, para compatibilidade sintática com corpora Protheus reais sem implementar comportamento que nenhum caso de uso exercitou.

Tabela de Opcodes

Status: 87 opcodes implementados na VM (0–86).

Documentação: docs/GUIA_D0_DESENVOLVEDOR_PARA_ADVPP.md seção 4.3 documenta todos os opcodes (0–86), nomes, descrições e efeito de stack.

Consulte ./docs/GUIA_D0_DESENVOLVEDOR_PARA_ADVPP.md para tabela detalhada e exemplos de cada opcode.

Resource Limits (Proteção contra DoS)

Status: AdvPP v2.0.3 impõe limites duros de recurso (CWE-400 Uncontrolled Resource Consumption):

Limite	Valor	Controle
Profundidade de recursão (parser)	1000	recursionDepth++
Tamanho de string	10 MB	len(s) > 10MB → erro
Tamanho de stack da VM	10000 frames	len(stack)
Call frames	5000	len(callStack)
Goroutines via StartJob	1000 concurrent	activeJobsCount
Tamanho de array	1M elementos	len(a) > 1M → erro
Timeout LLM Generate()	5 minutos	context.WithTimeout
Timeout I/O (arquivos)	30 segundos	context.WithTimeout
Timeout HTTP	30 segundos	http.Client.Timeout

Ultrapassar esses limites retorna erro capturável via Try/Catch ou falha gracefully (sem crash).

Versão da Extensão VS Code

Status: Extensão AdvPL/TLPP (marketplace + .vsix em Releases) segue a mesma versão do compilador — confira tools/vscode-advpl/package.json para o número exato publicado no momento (atualizado a cada release do advplc, não é reafirmado aqui para não ficar desatualizado).

Incluída: Compilador AdvPP embutido (linux-x64, linux-arm64, win32-x64, darwin-arm64); F5 e F9 para run/debug.